An Adaptive Hybrid Storage Engine with Autonomous Structural Metamorphosis for Cost-Aware Workload Optimization

Sujal Suryawanshi
Department of Computer Science and Engineering
IIT Patna, India
Email: sujal_2301ai26@iitp.ac.in

***Abstract*—Modern storage engines face a fundamental trade-off between write-optimized persistence and read-efficient analytical performance. Log-Structured Merge (LSM) trees provide high write throughput but suffer from read amplification, whereas B+ trees offer efficient range queries at the cost of slower writes.**

This paper presents an Adaptive Hybrid Storage Engine that resolves this trade-off through a cost-aware, workload-driven architecture. The system introduces an Autonomous Brain that models workload behavior as a stochastic process and dynamically triggers a structural metamorphosis, migrating hot data from a disk-based LSM-tree (RocksDB) to an in-memory B+ tree (LMDB) when the performance return-on-investment (ROI) is positive.

The decision process is grounded in a mathematical cost model using empirically measured constants, including write cost (ω_{LSM}) and indexing cost (α). To ensure production-grade stability, the system incorporates a stability hysteresis mechanism, RAM-aware safety constraints, and a dual-write concurrency protocol that guarantees strict linearizability and zero data loss during live migration.

Experimental evaluation across datasets up to 10^7 keys demonstrates up to $17.8\times$ real-world scan acceleration and theoretically up to $177\times$ improvement, while maintaining negligible write overhead ($\approx 4.2\,\mu s$). The results establish that adaptive structural transformation is a viable paradigm for next-generation autonomous database systems.

***Index Terms*—Hybrid Storage Engine, LSM Tree, B+ Tree, Workload Adaptation, Cost Model, RocksDB, LMDB, Linearizability**

I. Introduction

Modern data systems must simultaneously support high-throughput ingestion and low-latency analytical queries. Applications such as time-series databases, real-time monitoring systems, and recommendation engines operate under highly dynamic workloads, where the ratio of reads to writes can shift unpredictably over time.

Log-Structured Merge (LSM) trees, widely used in systems such as RocksDB, optimize write throughput by transforming random writes into sequential disk operations. However, this design introduces significant read amplification, making range queries inefficient due to multi-level storage and compaction overhead.

In contrast, B+ tree-based systems, such as LMDB, provide efficient range queries and predictable read latency due to their ordered structure and memory-mapped design. However, they incur higher write costs and reduced ingestion throughput.

This dichotomy leads to a long-standing systems problem: no single data structure can simultaneously optimize both write-heavy and read-heavy workloads.

In this paper, we propose an Adaptive Hybrid Storage Engine that dynamically combines LSM-tree and B+ tree paradigms using a cost-aware decision framework. The core innovation is the Autonomous Brain, which continuously monitors workload characteristics and models them as a stochastic process to distinguish true workload drift from transient noise.

Unlike prior approaches, our system is grounded in empirical cost modeling. We define the write cost of the LSM-tree as:

$$\omega_{LSM} = \frac{\text{Total Insert Time}}{\text{Total Records}} \tag{1}$$

and the indexing cost as:

$$\alpha = \frac{\text{Time to Iterate LSM}}{\text{Total Keys}} \tag{2}$$

These constants enable a cost-aware migration decision that ensures the benefit of switching to a B+ tree outweighs the cost of building it.

The system introduces a novel structural metamorphosis process that transforms the storage engine at runtime without downtime. This transformation is implemented using a dual-write concurrency protocol that guarantees strict linearizability and prevents data loss.

To ensure robustness in production environments, the system integrates:

- Stability hysteresis to prevent oscillatory behavior,
- RAM-aware constraints to avoid out-of-memory failures,
- A concurrency control mechanism based on `std::shared_mutex`.

The contributions of this paper are as follows:

- **Cost-Aware Adaptive Storage:** A novel hybrid architecture that dynamically switches between LSM-tree and B+ tree representations.
- **Mathematical Workload Modeling:** A stochastic framework using sliding windows and hysteresis to detect true workload drift.

- **Zero-Downtime Migration:** A dual-write protocol ensuring strict linearizability during structural transitions.
- **Production-Grade Stability:** Integration of RAM guards, hysteresis, and concurrency-safe state transitions.

Experimental results demonstrate that the proposed system significantly improves analytical query performance while preserving write efficiency, effectively resolving the read-write trade-off in modern storage systems.

II. Mathematical Model and Cost Formulation

The proposed system is grounded in a cost-aware framework that models workload behavior, migration cost, and system constraints using empirical measurements. This section formalizes the decision-making process of the Autonomous Brain.

A. Baseline Cost Model

To enable cost-aware adaptation, we define two fundamental constants measured during the baseline phase.

1) Write Cost (ω_{LSM}): The average latency of persisting a record in the LSM-tree is defined as:

$$\omega_{LSM} = \frac{T_{\text{insert}}}{N} \tag{3}$$

where T_{insert} is the total time to insert N records.

2) Indexing Cost (α): The unit cost of scanning and inserting a key into the in-memory index is:

$$\alpha = \frac{T_{\text{scan}}}{N} \tag{4}$$

This constant represents the cost of building the B+ tree index from the LSM-tree.

B. Workload as a Stochastic Process

We model the workload as a sequence of read ratios over time windows. Let x_i denote the read ratio in window i:

$$x_i = \frac{r_i}{r_i + w_i} \tag{5}$$

where r_i and w_i are the number of read and write operations in window i.

To mitigate noise caused by transient bursts, we define a sliding window of size W and apply a K-out-of-W decision rule.

C. Workload State Classification

The system classifies workload into three states:

$$S = \begin{cases} \text{READ_HEAVY}, & \text{if } \sum_{i=n-W}^{n} \mathbf{1}(x_i > T_{high}) \geq K \\ \text{WRITE_HEAVY}, & \text{if } x_n < T_{low} \\ \text{STABLE}, & \text{otherwise} \end{cases} \tag{6}$$

where:

- $T_{high} = 0.8$ (read-heavy threshold),
- $T_{low} = 0.4$ (write-heavy threshold),
- $K = 3$ (minimum windows required to confirm drift).

This formulation prevents false positives caused by short-lived workload fluctuations.

D. Migration Cost Model

The cost of building the in-memory index is proportional to the number of keys:

$$C_{build} = N \cdot \alpha \tag{7}$$

The expected benefit of migration depends on the improvement in scan latency. Let L_{base} denote the baseline scan latency using the LSM-tree, and L_{b+} denote the latency using the B+ tree.

The benefit per query is:

$$\Delta L = L_{base} - L_{b+} \tag{8}$$

The migration is triggered only if:

$$Q \cdot \Delta L > C_{build} \tag{9}$$

where Q is the expected number of future queries. This ensures that migration is only performed when the performance gain outweighs the cost.

E. RAM Constraint Model

To prevent out-of-memory failures, we define a memory constraint:

$$M_{req} = N \cdot (2 \cdot sizeof(uint64)) \cdot \gamma \tag{10}$$

where γ accounts for structural overhead in the B+ tree (typically $\gamma \approx 1.2$).

Migration is allowed only if:

$$M_{req} < M_{available} \tag{11}$$

This constraint is enforced at runtime through the RAM Guard.

F. Consistency and Linearizability

During migration, the system must ensure strict linearizability. Let the dataset be divided into two disjoint sets:

- D_{old}: Data captured in the snapshot,
- D_{new}: Data generated after migration begins.

The B+ tree index is consistent if:

$$I_{B+} = Scan(D_{old}) \cup Intercept(D_{new}) \tag{12}$$

To satisfy this condition, the system initiates dual-write before scanning begins, ensuring that all new writes are immediately reflected in both storage layers.

G. Dual-Write Overhead Constraint

Let the total write latency during migration be:

$$L_{total} = \omega_{LSM} + \omega_{B+} + \tau_{lock} \tag{13}$$

where τ_{lock} is the synchronization overhead.
The system enforces:

$$L_{total} < \Gamma \tag{14}$$

where Γ is a predefined latency threshold. Since the B+ tree resides in memory, ω_{B+} remains negligible, ensuring minimal performance degradation.

H. Correctness Guarantee

Theorem 1 (Linearizability During Migration): The proposed dual-write protocol guarantees that all operations appear to execute atomically in a globally consistent order.

Proof Sketch: Let S_{snap} denote the snapshot sequence number. All keys with sequence number $\leq S_{snap}$ are captured in D_{old}, while subsequent keys belong to D_{new}. Since dual-write is activated before scanning begins, all elements of D_{new} are immediately inserted into the B+ tree.

Thus, at any time t:

$$I_{B+} = D_{old} \cup D_{new}$$

which ensures completeness. Since writes are serialized through the dispatcher, a total order is preserved, satisfying linearizability.

III. System Architecture

The proposed Adaptive Hybrid Storage Engine is designed as a modular system that integrates persistent storage, in-memory indexing, and an autonomous decision layer. The architecture consists of three primary components: (i) the Dual Storage Layer, (ii) the Autonomous Brain, and (iii) the Consistency Dispatcher.

A. Dual Storage Layer

The system maintains two complementary storage structures:

- **Primary Store (LSM-tree):** Implemented using RocksDB, this layer serves as the durable source of truth and is optimized for high-throughput writes.
- **Secondary Index (B+ Tree):** Implemented using LMDB, this layer resides in memory and accelerates range queries through ordered indexing.

All write operations are first persisted in the LSM-tree to ensure durability. The B+ tree is dynamically constructed and activated only when the workload becomes read-heavy.

B. The StorageManager Abstraction

The core system logic is encapsulated in the `StorageManager` class, which provides a unified API for data operations. It manages:

- RocksDB and LMDB instances,
- Migration state flags (`is_lmdb_ready`, `is_migrating`),
- Concurrency control via `std::shared_mutex`,
- Telemetry logging and performance tracking.

The abstraction ensures that higher-level components remain independent of the underlying storage complexity.

C. Autonomous Brain (Workload Monitor)

The Autonomous Brain continuously observes system activity and determines when structural adaptation is required. It operates as a background thread that periodically:

1) Collects read and write operation counts,
2) Computes the read ratio over a sliding window,
3) Applies hysteresis-based decision rules,
4) Triggers migration or reversion based on workload state.

To avoid false positives, the system employs a K-out-of-W rule combined with a cooldown mechanism, ensuring that structural transitions occur only under sustained workload shifts.

D. Structural Metamorphosis Pipeline

The transition from LSM-tree to B+ tree occurs through a three-phase pipeline:

1) Phase 1: Snapshot Acquisition: The system captures a consistent snapshot of the LSM-tree using RocksDB's snapshot mechanism. This snapshot defines the frozen dataset D_{old}.

2) Phase 2: Dual-Write Activation: Immediately after snapshot creation, the system enables dual-write mode. All incoming writes are simultaneously applied to both the LSM-tree and the B+ tree. This ensures that the live dataset D_{new} is continuously synchronized.

3) Phase 3: Background Index Construction: A dedicated background thread iterates over the snapshot and inserts entries into the B+ tree. The process uses batch commits to improve efficiency and avoids overwriting keys already updated by live writes.

Once the build completes, the system atomically switches to the B+ tree as the primary read engine.

E. Consistency Dispatcher

The Consistency Dispatcher coordinates all read and write operations to ensure correctness during state transitions. It implements a Readers-Writer lock pattern:

- **Reads:** Allowed concurrently using shared locks.
- **Writes:** Acquire shared locks and update both storage layers when necessary.

This design guarantees strict linearizability, ensuring that all operations appear to execute in a globally consistent order.

F. RAM Guard and Stability Mechanisms

To ensure production-grade robustness, the system integrates two critical safeguards:

- **RAM Guard:** Before initiating migration, the system checks available system memory and aborts if usage exceeds a predefined threshold (e.g., 90%).
- **Stability Hysteresis:** A minimum residence time is enforced between transitions to prevent oscillatory behavior (thrashing) under fluctuating workloads.

G. Query Routing

During normal operation:

- Reads are served from RocksDB when the system is in baseline mode.
- Once migration completes, range queries are routed to the B+ tree for significantly faster execution.

The routing decision is performed dynamically based on the current system state.

H. Telemetry and Observability

The system includes a telemetry module that logs:

- Operation latency,
- Active storage engine,
- Timestamp of execution.

This data is exported to a CSV file and used for performance analysis and visualization.

IV. Implementation Details

This section describes the concrete implementation of the proposed system, focusing on storage configuration, data encoding, concurrency mechanisms, and engineering optimizations.

A. RocksDB Configuration (LSM-tree)

The primary storage layer is implemented using RocksDB, configured for high write throughput and efficient compaction.

Key configuration parameters include:

- **Leveled Compaction:** Optimized for balanced read/write performance.
- **Write Buffer Size:** Set to 64 MB to reduce flush frequency.
- **Block Cache:** An LRU cache (128 MB) is used to accelerate read operations.
- **Bloom Filters:** Enabled to reduce unnecessary disk reads.

For durability and performance balance, the system uses:

- `sync = false` (OS-level buffering for speed),
- `disableWAL = false` (ensuring crash recovery via Write-Ahead Logging).

B. LMDB Configuration (B+ Tree)

The secondary index is implemented using LMDB, which provides a memory-mapped B+ tree structure.

Key implementation choices:

- **Memory Map Size:** Configured to 1 GB to accommodate large datasets.
- **Batch Inserts:** Transactions are committed every 10,000 keys during index construction.
- **Read-Optimized Access:** Range queries use cursor-based traversal for sequential access.

Since LMDB operates directly on memory-mapped files, it provides near in-memory read performance while maintaining persistence.

C. Key Encoding Strategy

To ensure correct ordering of time-series data, keys are encoded using big-endian format.

On little-endian architectures (e.g., x86), raw 64-bit timestamps would be stored in reverse byte order, breaking range scan correctness. To address this, timestamps are transformed as:

$$K_{encoded} = \text{byteswap64}(timestamp) \tag{15}$$

This guarantees that lexicographical ordering of keys matches chronological ordering, ensuring correct range queries.

D. Concurrency Control

The system uses a Readers-Writer lock implemented via `std::shared_mutex` to manage concurrent access:

- **Shared Locks:** Used for concurrent read and write operations.
- **Exclusive Locks:** Used during index drop or structural transitions.

This design minimizes contention while ensuring thread-safe access to the B+ tree during migration and steady-state operation.

E. Dual-Write Implementation

During migration, each write operation is applied to both storage layers:

1) Write to RocksDB (persistent store),
2) Conditionally write to LMDB if migration is active or completed.

This ensures that newly inserted data is immediately reflected in the in-memory index, maintaining consistency across both structures.

F. Background Migration Thread

The index construction process is executed in a dedicated background thread. The thread:

- Acquires a consistent snapshot of RocksDB,
- Iterates over all keys using a sequential iterator,
- Inserts keys into LMDB in batches,

- Avoids overwriting newer values using a first-write-wins policy.

The use of detached threads ensures that migration does not block foreground operations.

G. RAM Guard Implementation

To prevent out-of-memory failures, the system implements a runtime memory check using the Linux `/proc/meminfo` interface.

The migration process is aborted if:

$$\text{Memory Usage} > 90\% \tag{16}$$

This ensures that index construction is only performed when sufficient system resources are available.

H. Telemetry Logging

The system includes a lightweight telemetry module that records performance metrics in real time.

Each log entry includes:

- Timestamp,
- Active storage engine (RocksDB or LMDB),
- Operation type (PUT or SCAN),
- Latency in microseconds.

The data is exported to a CSV file, enabling detailed performance analysis and visualization.

I. Failure Handling and Durability

To ensure robustness, the system was tested under failure scenarios:

- **Crash Recovery:** The Write-Ahead Log (WAL) ensures no data loss after abrupt termination.
- **Partial Migration Safety:** If migration is interrupted, RocksDB remains the authoritative source.

These mechanisms guarantee that the system maintains correctness under both normal and failure conditions.

V. Experimental Evaluation

This section evaluates the performance, scalability, and consistency of the proposed Adaptive Hybrid Storage Engine using real-world benchmarks.

A. Experimental Setup

All experiments were conducted on a Linux-based system using C++17 with RocksDB and LMDB libraries. The evaluation focuses on datasets ranging from 10^5 to 10^7 keys.

Workloads consist of mixed operations with approximately 85% range scans and 15% writes, simulating analytical query-heavy scenarios.

B. Baseline Cost Measurements

We first establish baseline constants required for the cost model:

TABLE I
BASELINE COST CONSTANTS

Metric	LSM-tree (RocksDB)	B+ Tree (LMDB)
Write Cost (ω)	$0.1631\,\mu s$	$0.1999\,\mu s$
Scan Cost (α)	$0.1135\,\mu s$	$0.0085\,\mu s$

These values confirm that LSM-trees are write-efficient, while B+ trees provide significantly faster scan performance.

TABLE II
OVERALL EXECUTION TIME COMPARISON

System	Total Time (ms)	Improvement
RocksDB (Baseline)	17807	–
Adaptive Engine	12570	29.4% faster

C. Head-to-Head Performance Comparison

We compare the proposed adaptive engine against a pure RocksDB baseline under identical workloads.

The adaptive system achieves a significant reduction in execution time due to dynamic query routing and optimized indexing.

D. Range Query Performance

Range scan latency is a critical performance metric for analytical workloads.

- LSM-tree scan latency: 0.7477 ms/scan
- B+ tree scan latency: 0.0419 ms/scan

This corresponds to a:

$$\text{Speedup} = \frac{0.7477}{0.0419} \approx 17.8\times \tag{17}$$

demonstrating substantial improvement in query performance after migration.

E. Migration Overhead

The system measures performance during the migration phase:

- LMDB index build time (1M keys): ≈ 300 ms
- Average scan latency during migration: $291.9\,\mu s$

These results show that the system maintains acceptable query latency even during structural transformation.

F. Write Path Overhead

To evaluate the cost of dual-write operations, we measure the average write latency:

$$L_{write} \approx 4.2359\,\mu s \tag{18}$$

This confirms that the additional overhead introduced by dual-write and locking mechanisms is negligible.

G. Concurrency and Consistency Validation

We perform stress testing with concurrent reads and writes during migration.

- Total inserts: 100,000 keys
- Total latency: 113 ms
- Verified keys: 150,000 (including live writes)

The system successfully maintains:

- **Strict linearizability**
- **Zero data loss**
- **No deadlocks or inconsistencies**

H. Adaptive Behavior Under Dynamic Workloads

We simulate workload shifts between read-heavy and write-heavy phases.

Key observations:

- Migration is triggered only under sustained read-heavy conditions.
- The system reverts to LSM-tree mode when writes dominate.
- Stability hysteresis prevents frequent oscillations.

The adaptive engine demonstrates stable and predictable behavior across varying workload conditions.

I. Memory Utilization

Peak memory usage during different phases:

- Ingestion phase: ≈ 75 MB
- Migration phase: ≈ 88 MB
- Dual-write phase: ≈ 96 MB
- Peak usage: ≈ 99 MB

The RAM Guard ensures that memory usage remains within safe limits.

VI. Conclusion and Future Work

This paper presented an Adaptive Hybrid Storage Engine that resolves the long-standing trade-off between write-optimized persistence and read-efficient analytical querying. By combining an LSM-tree (RocksDB) with an in-memory B+ tree (LMDB), the system dynamically adapts its internal structure based on real-time workload characteristics.

The core innovation lies in the Autonomous Brain, a cost-aware decision engine that models workload behavior as a stochastic process and triggers structural metamorphosis only when the expected performance gain exceeds the migration cost. Unlike static hybrid systems, this approach enables fully autonomous adaptation without manual tuning.

To ensure correctness and reliability, the system integrates a dual-write concurrency protocol that guarantees strict linearizability and zero data loss during migration. Additionally, production-grade safeguards such as stability hysteresis and RAM-aware constraints prevent oscillatory behavior and resource exhaustion.

Extensive experimental evaluation demonstrates that the proposed system achieves up to $17.8\times$ real-world scan acceleration and a 29.4% overall performance improvement, while maintaining negligible write overhead ($\approx 4.2\,\mu s$). These results validate that adaptive structural transformation is an effective strategy for optimizing mixed workloads.

A. Future Work

While the proposed system demonstrates strong performance and robustness, several directions remain for future exploration:

- **Distributed Extension:** Extending the architecture to distributed environments with sharding and replication support.
- **Predictive Workload Modeling:** Incorporating machine learning models to predict workload trends and proactively trigger migration.
- **Hardware-Aware Optimization:** Adapting the system for heterogeneous storage environments, including SSDs and persistent memory.
- **Adaptive Index Variants:** Exploring alternative in-memory index structures such as ART (Adaptive Radix Trees) or learned indexes.
- **Query-Aware Optimization:** Extending the system to support more complex query patterns beyond range scans.

The results of this work suggest that future database systems can move beyond static architectures toward fully self-adaptive, intelligent storage engines capable of optimizing themselves in real time.